Learning Low-Dimensional Structure from High-Dimensional Data

High-level question: Why can neural networks trained by stochastic gradient descent (SGD) perform so well on high-dimensional non-convex learning problems?

This talk: Thanks to feature learning, neural networks can adapt to low-dimensional structure in a sample-efficient way.

References:

"A Short Tutorial on the Computational Complexity of Deep Learning" (Misiakiewicz, 2025)

"Online Stochastic Gradient Descent on Non-Convex Losses from High Dimensional Inference" (Ben Arous et al., 2021)

"Neural Network Learns Low-Dimensional Polynomials with SGD near the Information-Theoretic Limit" (Lee et al., 2024)

"From Information to Generative Exponent: Learning Rate Induces Phase Transitions in SGD" (Tsiolis et al., 2025)

Target: Single-index model (GLM)

$x \sim N(0, I_d)$ $\quad y = f_*(x) = \sigma_*(\langle x, w_* \rangle)$, $\quad \sigma_* : \mathbb{R} \to \mathbb{R}$ polynomial of degree at most k, $\quad w_* \in \mathbb{S}^{d-1}$

Crucially, σ is unknown

Learner: Want to learn a function $g : \mathbb{R}^d \to \mathbb{R}$ from data $(x_i, y_i)_{i=1}^n$ such that the risk

$$R(g) = \mathbb{E}_{x \sim N(0, I_d)} \left[\frac{1}{2} (f_*(x) - g(x))^2 \right] \text{ is small.}$$

If not NNs, what's a standard way of dealing with this nonlinear problem?

- Kernel Regression

Note $f_* \in \mathcal{H} = \left\{ p : \mathbb{R}^d \to \mathbb{R} : p(x) = \sum_{\substack{\alpha_1, \dots, \alpha_d \geq 0 \\ \|\alpha\|_1 = k}} x_1^{\alpha_1} \cdots x_d^{\alpha_d} \right\}$

For $x, y \in \mathbb{R}^d$ define $K(x, y) = \langle x, y \rangle^k = \left(\sum_{j=1}^d x_i y_i \right)^k$

$\Rightarrow K(x,\cdot) \in \mathcal{H}$ and $\langle K(x,\cdot), K(y,\cdot)\rangle_{\mathcal{H}} = K(x,y)$

→ This means we have a fixed, non-learnable, representation for each $x \in \mathbb{R}^d$

→ It does not adapt to the inherent low dimensional structure of the data

Fit $g(\cdot) = \sum_{i=1}^{n} a_i K(x_i,\cdot) \in \mathcal{H}$

Then simply learn the coefficients a_i by least squares.

Problem: $\dim(\mathcal{H}) = \Theta(d^k)$

$\Rightarrow$ In the worst case, the kernel method needs $n = \Theta(d^k)$ to learn f_*.

- Two-Layer Neural Network

$$g(x) = \sum_{j=1}^{m} a_j \sigma_j(\langle x, w_j\rangle), \quad \underbrace{a_j \in \mathbb{R}}_{\text{second layer params}} \quad \underbrace{\sigma_j : \mathbb{R} \to \mathbb{R}}_{\text{activation fcns}} \quad \underbrace{w_j \in \mathbb{S}^{d-1}}_{\text{first layer params}}$$

For simplicity, I will focus on the case $m=1$, $\sigma = \sigma_*$.

But the following discussion will hold much more generally than that.

We want to study this from an algorithmic perspective: fitting the NN with SGD.

Though it is not an accurate reflection of practice, we consider online SGD for theoretical convenience.

We also focus on training the first layer parameter w and then optimizing a, but in practice, one would train both at the same time.

Goal: Train w to recover w_*.

Correlation loss: $\ell(w; x, y) = -y\sigma(\langle x, w\rangle)$ $\quad \nabla_w \ell = -y\sigma'(\langle x, w\rangle)x$

This loss is just the cross term in the L_2 loss.

(Spherical) Online SGD for w:

Initialize $w^{(0)} \sim \mathrm{Unif}(\mathbb{S}^{d-1})$

→ If d is large, then $\langle w_*, w^{(0)} \rangle \approx d^{-1/2}$ w.h.p.

For $t = 0, 1, 2, \dots, T$

Sample $x^{(t)}, y^{(t)} \overset{iid}{\sim}$ Target

$$w^{(t+1)} \leftarrow w^{(t)} + \gamma y \sigma'(\langle x, w^{(t)} \rangle) \underbrace{(I_d - w^{(t)} w^{(t)\top})}_{P_{w^{(t)}}^{\perp}} x$$

$$w^{(t+1)} \leftarrow \frac{w^{(t+1)}}{\|w^{(t+1)}\|}$$

Analysis: Track $\mathbb{E}[\langle w_*, w^{(t+1)} \rangle | w^{(t)}] = \langle w_*, w^{(t)} \rangle + \gamma w_*^\top \mathbb{E}_{x^{(t)}}[\underbrace{y^{(t)}}_{= \sigma(\langle x^{(t)}, w_* \rangle)} \sigma'(\langle x^{(t)}, w^{(t)} \rangle) P_{w^{(t)}}^{\perp} \underbrace{x^{(t)}}_{\sim N(0, I_d)}]$

Idea: Hermite expansion $\sigma_*(z) = \sum_{j=0}^{\infty} c_j \mathrm{He}_j(z)$ $\quad c_j = \mathbb{E}_{z \sim N(0,1)}[\sigma_*(z) \mathrm{He}_j(z)]$

$$\mathbb{E}_{x \sim N(0, I_d)}[\mathrm{He}_j(\langle x, w_* \rangle) \mathrm{He}_k(\langle x, w \rangle)] = k! \langle w_*, w \rangle^k \delta_{j=k}$$

$$\Rightarrow w_*^\top \mathbb{E}_x[\sigma_*(\langle x, w_* \rangle) \sigma(\langle x, w \rangle) P_w^{\perp} x]$$

$$= w_*^\top \mathbb{E}_x\left[\left(\sum_{j=0}^{\deg(\sigma_*)} c_j \mathrm{He}_j(\langle x, w_* \rangle)\right)\left(\sum_{j=0}^{\deg(\sigma_*)} c_j \mathrm{He}_j(\langle x, w \rangle)\right) P_w^{\perp} x\right] \simeq \underbrace{\langle w_*, w \rangle^{p-1}}_{\le 1} + \text{higher order terms},$$

where $p = \underbrace{\mathrm{IE}(\sigma)}_{\text{"information exponent" (Ben Arous et al., 2021)}} := \min\{j : c_j \neq 0\}$.

For σ any non-Hermite polynomial of degree k, $p < k$.

One can show $\mathbb{E}[\langle w_*, w^{(t+1)} \rangle | w^{(t)}] \ge \langle w_*, w^{(t)} \rangle + C_\gamma \langle w_*, w^{(t)} \rangle^{p-1}$ if $\gamma \lesssim d^{-\frac{p}{2}}$.

Solving the recurrence with $\langle w_*, w^{(0)} \rangle = d^{-\frac{1}{2}}$ yields $\mathbb{E}[\langle w_*, w^{(T)} \rangle] \simeq T \gamma d^{\frac{p-2}{2}}$

⇒ We can get $\mathbb{E}[\langle w_*, w^{(T)} \rangle] = \Theta(1)$ after $T = \Theta(d^{p-1})$ with "correct" choice of γ. 

We can remove the $\mathbb{E}[\cdot]$ and keep the same statement with high probability by controlling the SGD noise via a martingale argument. From there, recovery of w_* to arbitrary accuracy proceeds quickly.

But can we be more sample-efficient?

In practice, SGD goes over the data multiple times (epochs).

Theory cannot yet cover this but it can analyze some form of data reuse.

Batch Reuse SGD (Lee et al., 2024)

Initialize $w^{(0)} \sim \mathrm{Unif}(\mathbb{S}^{d-1})$

For $t = 0, 1, 2, \dots, T$

Sample $x^{(t)}, y^{(t)} \overset{iid}{\sim} \text{Target}$

$$\tilde{w}^{(t)} \leftarrow w^{(t)} + \eta y^{(t)} \sigma'(\langle x^{(t)}, w^{(t)} \rangle) P_{w^{(t)}}^{\perp} x^{(t)}$$

$$w^{(t+1)} \leftarrow w^{(t)} + \gamma y^{(t)} \sigma'(\langle x^{(t)}, \tilde{w}^{(t)} \rangle) P_{w^{(t)}}^{\perp} x^{(t)}$$

} Two gradient steps with different step sizes

$$w^{(t+1)} \leftarrow \frac{w^{(t+1)}}{\|w^{(t+1)}\|}$$

Thm: Batch Reuse SGD has $\tilde{\Theta}(d)$ sample complexity whenever σ_* is polynomial, $\gamma \asymp d^{-1}$, $\eta \asymp d^{-1}$.

Why? Implicit label transformation

$$w^{(t+1)} = w^{(t)} + \gamma y^{(t)} \sigma'\left(\langle x^{(t)}, w^{(t)} + \eta y^{(t)} \sigma'(\langle x^{(t)}, w^{(t)} \rangle) P_{w^{(t)}}^{\perp} x^{(t)} \rangle\right) P_{w^{(t)}}^{\perp} x$$

$$= w^{(t)} + \left[\gamma \sigma'(\langle x^{(t)}, w^{(t)} \rangle) \sigma_*(\langle x^{(t)}, w_* \rangle) + \gamma \sum_{j=2}^{\deg(\sigma)} \frac{(\eta d)^{j-1} (\sigma^{(j)}(\langle x, w^{(t)} \rangle)(\sigma'(\langle x, w^{(t)} \rangle))^{j-1}}{(j-1)!} \left[\sigma_*(\langle x, w_* \rangle)\right]^j \right] P_{w^{(t)}}^{\perp} x$$

Lem: For σ polynomial, there exists $q \in \mathbb{N}$ s.t. $\mathbb{E}(\sigma^q) \le 2$.

Note: We only get $\Theta(d)$ sample complexity if γ, η are large enough.

Tsiolis, Mousavi-Hosseini, Erdogdu (2025): Precise characterization of sample complexity dependence on γ, η for a general class of gradient-based methods, including batch reuse SGD.

Define $\mu_i(\eta) := \sum_{j=1}^{r} \frac{(\eta d)^{j-1}}{(j-1)} c_{i-1}(\sigma^{(k)}(\sigma')^{k-1}) c_i(\sigma_*^j)$.

Then the sample complexity, in terms of γ, η is

$$T(\eta) = \min_{1 \le i \le r} \tilde{\Theta}\left(\gamma^{-1} (\eta d)^{-(i-1)} d^{\max\left\{\frac{IE(\sigma_*^i)}{2} - 1, 0\right\}} \right), \quad \gamma \lesssim \max_{1 \le i \le r} (\eta d)^{-1} d^{-\max\left\{\frac{IE(\sigma_*^i)}{2}, 0\right\}}$$

Key Implication: Phase Transition